Week 5, Day 4: Docker in Depth, CI/CD with GitHub Actions and Jenkins, and Union Find

Prepared for Upendra Kumar Sharma | 4 hours, plus an optional 90-minute lab | Main topic: production-quality Docker images, networking, CI/CD pipelines | DSA: Union find (disjoint set union)

Today's Schedule

- **0:00 to 0:50** Study Dockerfiles, multi-stage builds, image hygiene and the JVM in containers (sections 1 and 2).
- **0:50 to 1:20** Study Docker networking, volumes and Compose (section 3).
- **1:20 to 2:30** Study CI/CD concepts, GitHub Actions and Jenkins (sections 4 to 6).
- **2:30 to 3:30** DSA: the union find template and the 2 problems in section 8. Allow 25 minutes each.
- **3:30 to 4:00** Answer the interview questions aloud (section 9) and add 10 flashcards.
- **Optional (90 min):** the hands-on lab in section 7.

Your resume lists Docker, Jenkins and CI/CD pipelines. After today you should be able to describe, from real use, a pipeline that takes a commit to a deployed container, including the security and rollback parts.

1. Docker Fundamentals Worth Knowing Cold

- **Image:** a read-only, layered template. **Container:** a running instance of an image with a thin writable layer. Containers share the host kernel, so they start fast and are lighter than virtual machines, but isolate less.
- **Layers and caching:** each Dockerfile instruction creates a layer, and Docker reuses a cached layer if the instruction and everything before it are unchanged. **Order instructions from least to most frequently changing.** Copy the dependency file (pom.xml) and resolve dependencies **before** copying source code, so a code change does not re-download every dependency.
- **Key instructions:** FROM (base image), WORKDIR, COPY (prefer it to ADD), RUN, ENV (runtime variable) and ARG (build-time variable), EXPOSE (documentation only, it does not publish a port), USER, HEALTHCHECK, and ENTRYPOINT with CMD.
- **ENTRYPOINT vs CMD:** ENTRYPOINT is the fixed executable and CMD gives default arguments that users can override. **Use the exec form** (a JSON array), so the Java process is PID 1 and **receives termination signals** (SIGTERM) for graceful shutdown.

The shell form wraps your process in a shell that may not forward signals.

- **.dockerignore:** keep target folders, .git, local config and secrets out of the build context. It speeds up builds and prevents accidental leaks.

2. Production Images

Multi-stage builds: use a heavy **build stage** (JDK and Maven) and copy only the jar into a slim **runtime stage** (JRE). The final image has no build tools or source code, so it is smaller, faster to pull, and has fewer vulnerabilities.

```
# syntax=docker/dockerfile:1
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
RUN --mount=type=cache,target=/root/.m2 mvn -q dependency:go-offline
COPY src ./src
RUN --mount=type=cache,target=/root/.m2 mvn -q package -DskipTests

FROM eclipse-temurin:21-jre
WORKDIR /app
RUN useradd -r -u 1001 appuser
COPY --from=build /app/target/*.jar app.jar
USER appuser
ENV JAVA_TOOL_OPTIONS="-XX:MaxRAMPercentage=75"
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

- The **BuildKit cache mount** keeps the Maven repository between builds, so even a changed pom.xml does not start from nothing.
- **Layered jars:** Spring Boot can split the jar into layers (dependencies, loader, application classes), so a code change only rebuilds the small application layer. Use the layer extraction mode of the jar (the option name differs between Spring Boot versions, so check the documentation).
- **Alternatives to a hand-written Dockerfile: Cloud Native Buildpacks** (mvn spring-boot:build-image, which uses Paketo and sets sensible JVM memory options) and **Jib** (builds an optimized image without a Docker daemon). Know that they exist and when a Dockerfile still gives you more control.
- **Base image choice:** a slim Temurin JRE image is a safe default. **Distroless** images have no shell or package manager, which shrinks the attack surface, but make debugging harder. **Alpine** images are small but use musl instead of glibc, which can cause problems with native libraries. A custom runtime built with **jlink** can shrink the image further.

Hygiene and security checklist

- **Run as a non-root user.** Consider a **read-only root filesystem** and dropping Linux capabilities.
- **Never put secrets in the image.** Anything in ENV, ARG or a COPY becomes part of a layer and can be read from the image history. Inject secrets at runtime (environment variables from a secrets manager, mounted files).
- **Pin base image versions** (and ideally digests). Never deploy a tag named latest, because you can no longer say what is running.
- **Scan images** for known vulnerabilities (Trivy, Grype, Docker Scout, or ECR scanning), and fail the pipeline on high severity findings that have fixes. **Rebuild regularly** to pick up patched base images.
- **Keep images small** and keep one process per container.
- **Sign images and generate an SBOM** (a software bill of materials) in mature pipelines. Name them as advanced practice.

The JVM inside a container

- Modern JVMs are **container-aware**. They read the container's memory and CPU limits. Set the heap as a **percentage** of the limit (MaxRAMPercentage, 75 is typical), so the process leaves room for metaspace, threads and native memory. Setting the heap equal to the whole limit causes the kernel to kill the container (out-of-memory kill).
- **CPU limits change the number of processors the JVM sees**, which sizes garbage collector threads and common pools. A very small CPU limit (such as half a core) can make start-up and GC slow. Set limits deliberately.
- Make sure the application handles **SIGTERM** gracefully (Spring Boot graceful shutdown) and that the platform's stop timeout is longer than your shutdown timeout.
- A container needs a **health check**. The JRE image has no curl, so rely on the orchestrator's check (ECS, Kubernetes, or the Compose healthcheck using a tool that exists in the image).

3. Networking, Volumes and Compose

Network drivers

- **bridge (the default):** containers on one host get private addresses and reach the outside through NAT. On a **user-defined bridge network** containers resolve each other **by name** through built-in DNS. The default bridge does not do this.
- **host:** the container shares the host's network stack, with no isolation or port mapping.
- **none:** no networking. **overlay:** spans several hosts (Swarm).

- **Publishing ports:** -p 8080:8080 maps a host port to a container port. Only published ports are reachable from outside the host.

The classic mistake: inside a container, **localhost means the container itself**. An application that connects to localhost:5432 will not find a database in another container. Use the **service name** (Compose or a user-defined network), or host.docker.internal to reach the host machine during development.

Volumes

- **Named volumes** are managed by Docker and persist data beyond the container, which is the right choice for databases. **Bind mounts** map a host folder (handy for development, but they depend on the host's paths and permissions). **tmpfs** is in-memory storage.
- Container filesystems are **ephemeral**. Anything written there disappears when the container is replaced, so state belongs in volumes or external services.

Compose (as in your project)

- Describes a multi-container application in one file. Compose creates a **default network** where services find each other by name.
- Use **healthcheck** with **depends_on: condition: service_healthy** so that applications start only after PostgreSQL is ready. Plain depends_on only waits for the container to start.
- Use **env_file** or environment variables for configuration, **profiles** to start optional services, and override files for local differences.
- **Useful commands:** docker compose up --build, docker compose logs -f service, docker compose exec service sh, docker compose down -v (the -v also deletes volumes and your data), docker stats, docker inspect, and docker system prune to reclaim disk.
- **Resource limits:** set memory and CPU limits so one container cannot starve the host, and so that the JVM sees realistic values.

4. CI/CD Concepts

Definitions

- **Continuous Integration (CI):** developers merge small changes often, and every change is **built and tested automatically**, so problems are found within minutes.
- **Continuous Delivery:** every change that passes the pipeline is **ready to release**, with a manual approval for production.
- **Continuous Deployment:** every passing change **goes to production automatically**.

A typical pipeline for your services

1. **Checkout** and restore dependency caches.
2. **Build** and run **unit tests**.
3. **Static checks:** formatting and style (Checkstyle), bug patterns (SpotBugs), code quality (SonarQube), and dependency and secret scanning.
4. **Integration tests** with Testcontainers.
5. **Package** and **build the image**, tagged with the **git commit SHA**.
6. **Scan the image**, then **push** it to the registry (ECR).
7. **Deploy to a staging environment** and run smoke tests.
8. **Manual approval** (for continuous delivery), then **deploy to production**.
9. **Verify** with health checks and metrics, and **roll back** automatically or by a button if checks fail.

Principles to quote

- **Build once, deploy many:** build the artifact once and **promote the same image** through staging and production. Configuration is injected per environment. Never rebuild for production.
- **Fast feedback:** keep the main pipeline under about 10 minutes. Cache dependencies, run tests in parallel, and fail fast.
- **Pipeline as code:** the pipeline definition lives in the repository and is reviewed like any code.
- **Trunk-based development** with short-lived branches and small pull requests, protected main branch, required status checks and code review.
- **Secrets:** never hard-code or print them. Use the CI system's secret store, and for cloud access prefer **short-lived credentials through OIDC federation** instead of long-lived access keys.
- **Database migrations** are part of the release. Make them **backward compatible** (the expand and contract approach: add the new column first, deploy code that handles both, then remove the old one later), so a rollback of the application is safe.

Deployment strategies

- **Rolling:** replace instances gradually. Simple, with a mix of old and new versions during the rollout.
- **Blue/green:** run the new version next to the old one and switch traffic at once. Instant rollback by switching back, at the cost of double capacity for a while.
- **Canary:** send a small share of traffic to the new version, watch the metrics, and increase gradually.
- **Feature flags** separate deploying code from releasing a feature.

Measuring delivery (DORA metrics): deployment frequency, lead time for changes, change failure rate, and time to restore service.

5. GitHub Actions

Vocabulary: a **workflow** (a YAML file in .github/workflows) is triggered by **events** (push, pull request, schedule, manual). It has **jobs** that run on **runners**, each made of **steps** that run commands or reusable **actions**. Other features: **secrets**, **environments** with required reviewers and their own secrets, **matrix** builds, **caching**, **artifacts**, and **reusable workflows**.

Deploying to AWS without stored keys (OIDC): you create an **IAM role** in AWS that trusts GitHub's OIDC provider (restricted to your repository and branch), and the workflow **assumes that role** to get temporary credentials. There are no long-lived access keys in GitHub secrets, which is the secure and now standard way.

```
name: CI-CD
on:
  push:
    branches: [main]
  pull_request:

permissions:
  id-token: write       # needed for OIDC
  contents: read

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: 21
          cache: maven
      - run: mvn -B verify

  image:
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4
```

```
        with:
          role-to-assume: arn:aws:iam::123456789012:role/github-deploy
          aws-region: ap-south-1
      - id: ecr
        uses: aws-actions/amazon-ecr-login@v2
      - run: |
          IMAGE=${{ steps.ecr.outputs.registry }}/order-service:${{
github.sha }}
          docker build -t $IMAGE services/order-service
          docker push $IMAGE

  deploy:
    needs: image
    runs-on: ubuntu-latest
    environment: production      # can require a manual approval
    steps:
      - run: echo deploy the image tagged with the commit SHA to ECS here
```

- The account id, role name and region are placeholders. Check the current major versions of each action before you use them.
- The deploy step can use the ECS deploy action (register a new task definition revision with the new image, update the service, and wait for it to become stable), and the ECS **deployment circuit breaker** rolls back a bad release.
- **Quality gates** to add: a coverage threshold with JaCoCo, dependency updates and alerts (Dependabot), code scanning (CodeQL), secret scanning, and an image scan with Trivy. Protect main so that merging needs green checks and a review.

6. Jenkins (Because Your Resume Lists It)

- **What it is:** a self-hosted automation server with a huge plugin ecosystem. A **controller** schedules work, and **agents** (nodes) execute it, which can be permanent machines or containers started on demand.
- **Pipeline as code:** a **Jenkinsfile** in the repository, in **declarative** syntax (the common form) or scripted Groovy. A **multibranch pipeline** discovers branches and pull requests automatically. **Webhooks** from your Git host trigger builds. **Shared libraries** reuse pipeline code.
- **Credentials:** kept in Jenkins' credential store and exposed to steps with a binding. Never echo them.

```
pipeline {
    agent any
    tools { maven 'Maven3'; jdk 'JDK21' }
    stages {
        stage('Build') {
            steps { sh 'mvn -B -DskipTests package' }
        }
        stage('Test') {
            steps { sh 'mvn -B verify' }
            post { always { junit '**/target/surefire-reports/*.xml' } }
        }
        stage('Image') {
            when { branch 'main' }
            steps { sh 'docker build -t order-service:${GIT_COMMIT} .' }
        }
    }
}
```

- **Strengths:** full control, runs inside your network, very flexible. **Weaknesses:** you maintain and patch the server, plugins can conflict or have vulnerabilities, and configuration can drift unless it is defined as code.
- **Jenkins vs GitHub Actions:** Actions is hosted, needs no server, and integrates tightly with GitHub. Jenkins suits organizations that need self-hosting or already have heavy Jenkins investment. Say which you have actually used, and be honest about it.
- **Security note:** giving a build access to the Docker socket is effectively root access to the host. Use dedicated build agents, rootless or daemonless builders (Kaniko, Buildah, Jib), and isolation.

Infrastructure as code: define networks, load balancers, ECS services and databases in **Terraform**, **AWS CDK** or **CloudFormation**, so environments are repeatable and reviewable, and you can **tear them down and recreate them** safely. Terraform keeps a **state** file (store it remotely with locking) and shows a **plan** before it changes anything. For Day 6, defining your ECS and ALB setup in code is a strong stretch goal.

7. Optional Hands-On Lab (90 Minutes)

1. **Improve a Dockerfile.** Apply the multi-stage build, cache mounts, a non-root user and a .dockerignore. Record the image size and the build time before and after, and for a second build after a one-line code change.
2. **Scan.** Scan your image with Trivy (run it from its container image). Read the findings, fix one by changing the base image version, and scan again.

3. **Networking.** Create a user-defined network, start two containers and call one from the other by name. Then reproduce the localhost mistake and fix it.
4. **GitHub Actions.** Extend your project's CI so that on main it builds the image and scans it. Optionally push to ECR using an OIDC role. Turn on branch protection requiring the checks.
5. **Jenkins.** Run Jenkins in a container, create a multibranch pipeline from a Jenkinsfile, and make it build and test one service. Note what you had to configure by hand.
6. **Rollback drill.** Deploy an image that fails its health check, observe the result, and roll back to the previous tag. Write down how long it took.
7. Delete any AWS resources you created, and check your billing page tomorrow.

8. DSA: Union Find, or Disjoint Set Union (2 Problems)

What it is: a structure that tracks which elements belong to the same group, with two operations: **find** (which group is this element in, identified by a representative) and **union** (merge two groups). It is perfect for **counting connected components, detecting cycles in undirected graphs, Kruskal's minimum spanning tree**, and dynamic connectivity.

Two optimizations that make it fast

- **Path compression:** during find, point nodes closer to the root.
- **Union by size (or rank):** attach the smaller tree under the larger one.
- Together they give an amortized cost of about **O(alpha(n))** per operation, where alpha is the inverse Ackermann function, effectively constant.

```
class UnionFind {
    private final int[] parent;
    private final int[] size;
    int components;

    UnionFind(int n) {
        parent = new int[n];
        size = new int[n];
        components = n;
        for (int i = 0; i < n; i++) {
            parent[i] = i;
            size[i] = 1;
        }
    }

    int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]];     // path halving
```

```
            x = parent[x];
        }
        return x;
    }

    boolean union(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return false;            // already connected
        if (size[ra] < size[rb]) { int t = ra; ra = rb; rb = t; }
        parent[rb] = ra;
        size[ra] += size[rb];
        components--;
        return true;
    }
}
```

Problem 1: Number of Provinces

- **The problem:** an n by n matrix where isConnected[i][j] is 1 if cities i and j are directly connected. A province is a group of cities connected directly or indirectly. Count the provinces.
- **Idea:** start with n separate groups. For every pair marked connected, union them. The answer is the remaining number of components. (A DFS or BFS over the matrix also works in O(n squared).)
- **Complexity:** O(n squared x alpha(n)) time, O(n) space.
- **Edge cases:** one city, no connections (n provinces), a fully connected matrix (1 province). Ignore the diagonal.

```
public int findCircleNum(int[][] isConnected) {
    int n = isConnected.length;
    UnionFind uf = new UnionFind(n);
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (isConnected[i][j] == 1) uf.union(i, j);
        }
    }
    return uf.components;
}
```

Problem 2: Redundant Connection

- **The problem:** a graph that started as a tree on nodes 1 to n had one extra edge added. Given the edges, return the edge that can be removed to leave a tree. If several answers exist, return the one that appears last in the input.
- **Idea:** process the edges in order and **union** their endpoints. The first edge whose endpoints are **already connected** closes a cycle, so it is the redundant one. Since exactly one extra edge exists, that edge is the answer.
- **Complexity:** O(n x alpha(n)) time, O(n) space.
- **Edge cases:** the redundant edge appearing early or last, a cycle of length 3, nodes numbered from 1 (allocate n + 1 slots).
- **Connection to systems:** the same cycle check detects circular dependencies in a build or a configuration graph.

```
public int[] findRedundantConnection(int[][] edges) {
    UnionFind uf = new UnionFind(edges.length + 1);
    for (int[] edge : edges) {
        if (!uf.union(edge[0], edge[1])) return edge;
    }
    return new int[0];
}
```

Pattern to remember: when a problem merges groups and asks about connectivity, counts of groups, or the first edge that forms a cycle, reach for union find. Related problems: Number of Islands (grid version), Accounts Merge, and Graph Valid Tree.

9. Interview Questions to Answer Aloud

1. How does Docker layer caching work, and how should you order a Dockerfile to use it?
2. What is a multi-stage build, and what do you gain from it?
3. ENTRYPOINT vs CMD, and why use the exec form?
4. List the steps to make a Docker image secure. Where can secrets leak?
5. How does the JVM behave inside a container, and how do you configure its memory?
6. Why does an application inside a container fail to reach a database at localhost, and how do you fix it?
7. Bridge, host and overlay networks: what is each for? How do containers find each other by name?
8. Why is depends_on not enough to wait for a database, and what do you use instead?
9. Explain CI, continuous delivery and continuous deployment, and describe the pipeline you have used or built.

L0. What does build once, deploy many mean, and why does it matter?

L1. Compare rolling, blue/green and canary deployments. How do you roll back safely, including database changes?

L2. How does GitHub Actions authenticate to AWS without stored keys?

L3. Jenkins vs GitHub Actions: what are the trade-offs? What is a Jenkinsfile, and what are agents?

L4. What quality and security gates would you put in a pipeline?

10. End-of-Day Checklist

- [ ] I can write a production Dockerfile from memory (multi-stage, cache-friendly, non-root, exec-form entrypoint) and explain each choice.
- [ ] I can list the security hygiene steps for images and name an image scanner.
- [ ] I can explain the JVM memory settings in containers and Docker networking basics, including the localhost mistake.
- [ ] I can describe a full CI/CD pipeline, deployment strategies, and how rollback works.
- [ ] I can explain OIDC deployment from GitHub Actions to AWS, and read a Jenkinsfile.
- [ ] I wrote the union find template and solved Number of Provinces and Redundant Connection.
- [ ] (Optional) I improved my images, scanned them, and ran a rollback drill.
- [ ] I wrote at least 10 flashcards from questions I got wrong.

Tomorrow (Week 5, Day 5): Kubernetes (Pods, Deployments, Services, ConfigMaps and Secrets, probes, autoscaling), plus shortest path problems with Dijkstra's algorithm.